Seminar 07

React Hook Form and State Managment

PB138 - Basics of Web Development

Agenda

1. **Before RHF**: Controlled vs. uncontrolled inputs
2. **RHF Core Concepts**: `useForm`, register, submit, and Zod typing
3. **Advanced RHF Patterns**: `watch`, `useWatch`, and dynamic fields
4. **Shadcn/UI + RHF**: Form integration and field anatomy
5. **State Management Foundations**: Why, when, and trade-offs
6. **Global State Options**: Redux, Context API, Zustand, Jotai/Recoil
7. **Server and URL State**: TanStack Query and TanStack Router
8. **Persistence**: Web Storage

Life Before RHF: Controlled Inputs

- In React, we are used to `useState` being the "Single Source of Truth".
- **Problem**: Every single keystroke changes the state, triggering a re-render of the entire component and all its children.

```
// Standard controlled approach
const [value, setValue] = useState("");

return (
  <form>
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
    <p>Current value: {value}</p>
  </form>
);
```

- With large forms containing dozens of fields, this causes noticeable lag in the application.

Uncontrolled Inputs (useRef)

- Alternative: Let the DOM hold the value itself and extract it only upon submission.
- We use the `useRef` hook.

```
const inputRef = useRef<HTMLInputElement>(null);

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  console.log("Submitted:", inputRef.current?.value);
};

return (
  <form onSubmit={handleSubmit}>
    <input ref={inputRef} />
    <button>Submit</button>
  </form>
);
```

- **Advantage**: No re-renders while typing.
- **Disadvantage**: Manual validation and error handling are a nightmare.

React Hook Form (RHF)

- RHF combines the best of both worlds.
- It uses **uncontrolled inputs** for high performance but provides a **synchronous API** for validation and error handling.
- Main tool: the `useForm` hook.

Input Registration (register)

- The `register` function creates a bridge between RHF and your input.
- Under the hood, it assigns a `ref`, `name`, `onChange`, and `onBlur` to the input.

```
const { register, handleSubmit } = useForm();

// "title" is the key where the value will be found during submission
return (
  <form>
    <input {...register("title")} />
  </form>
);
```

Form Submission (handleSubmit)

- RHF provides a wrapper for your submission function.
- `handleSubmit` automatically calls `e.preventDefault()` and passes the data to you as a clean JS object.

```
const { register, handleSubmit } = useForm<CreateProduct>();

const onSubmit: SubmitHandler<CreateProduct> = (values) => {
  console.log("Valid data:", values);
};

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register("title")} />
    <button type="submit">Submit</button>
  </form>
);
```

TypeScript, Zod & SubmitHandler

- Connecting Zod and RHF ensures that the data in `onSubmit` has the correct type.
- **SubmitHandler<T>** is a type for your function that ensures the `data` argument exactly matches your schema.

```
const schema = z.object({
  title: z.string().min(3, "Title too short"),
});

type CreateProduct = z.infer<typeof schema>;

const ProductForm = () => {
  const { register, handleSubmit, formState: { errors } } = useForm<CreateProduct>({
    resolver: zodResolver(schema) // Connects validation
  });

  const onSubmit: SubmitHandler<CreateProduct> = (data) => {
    console.log(data.title); // TS knows this is a string
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("title")} />
      {errors.title && <p>{errors.title.message}</p>}
      <button type="submit">Submit</button>
    </form>
  );
};
```

Task 1: Basic Form

- Create a simple component `SimpleProductForm.tsx`.
- Define a Zod schema for a product: `name` (string, min. 2 chars) and `price` (number).
- Initialize `useForm` with this schema and the `zodResolver`.
- Create an HTML form with two inputs for these fields.
- Implement an `onSubmit` function that logs valid data to the console.

Concept: watch (Method from useForm)

- The easiest way to track values in real-time directly within the main form component.
- **Mechanism**: Every change to a watched field triggers a **re-render of the entire form** (all inputs and components within it).
- **When to use**:
  - Small forms (few fields) where performance isn't critical.
  - Conditional field rendering within the same component.
- **Disadvantage**: Everything re-renders with every keystroke. With 50 inputs, the app will start to lag.

```
const { register, watch } = useForm<CreateStudent>();
const isInternational = watch("isInternational");

return (
  <form>
    <input type="checkbox" {...register("isInternational")} />
    {isInternational && <p>Displaying extra fields for international students...</p>}
  </form>
);
```

Concept: useWatch (Separate Hook)

- An advanced tool for **isolating re-renders** in large forms.
- **Mechanism**: The value is retrieved directly from the RHF state, but **only the component** where `useWatch` is called will re-render.
- **When to use**:
  - Large and complex forms.
  - Isolated components (e.g., profile preview, live price calculator) that need data while the rest of the form stays "quiet."
- **Advantage**: Dramatically better performance and smoothness.

```
function TotalPrice({ control }: { control: Control<FormData> }) {
  const amount = useWatch({ control, name: "amount", defaultValue: 0 });
  return <div>Total: {amount * 1.2} €</div>;
}
```

Task 2: Dynamic Form

- Extend the form from Task 1 with interactivity.
- Add a `hasDiscount` field (boolean) to the schema.
- Add a `discountCode` field (string, optional).
- **Condition**: The input for `discountCode` should appear in the UI **only if** the `hasDiscount` checkbox is checked.

Dynamic Fields (useFieldArray)

- Used when you don't know the number of input fields in advance (e.g., "Add another phone number").
- **Key Features**:
  - Provides methods like `append`, `prepend`, `remove`, and `move`.
  - Each row has a unique `id` generated by the library.
- **Performance**: Optimized to re-render only the necessary parts when adding or removing items.

```
const { fields, append, remove } = useFieldArray({
  control, // 'control' comes from the useForm() hook
  name: "emails"
});

return (
  <form>
    {fields.map((field, index) => (
      <div key={field.id} className="flex gap-2 mb-2">
        <input
          {...register(`emails.${index}.value`)} // 'register' comes from the useForm() hook
          className="border p-1"
        />
        <button type="button" onClick={() => remove(index)}>
          Remove
        </button>
      </div>
    ))}
    <button
      type="button"
      onClick={() => append({ value: "" })}
      className="bg-blue-500 text-white p-2 rounded"
    >
      Add Email
    </button>
  </form>
);
```

React Hook Form: Explicit Controller Integration

- Using the native `<Controller />` instead of high-level abstractions gives us full control over the field lifecycle.
- **Explicit Accessibility**: Manual but reliable linking of `htmlFor` and `id`.
- **State Visibility**: Direct access to `fieldState` (invalid, dirty, error) for custom styling.
- **Predictability**: No hidden "magic" context – you decide exactly how the input behaves.

```
<Controller
  name="username"
  control={form.control}
  render={({ field, fieldState }) => (
    <Field data-invalid={fieldState.invalid}>
      <FieldLabel htmlFor={field.name}>Username</FieldLabel>
      <FormControl>
        <Input
          {...field}
          id={field.name}
          aria-invalid={fieldState.invalid}
          placeholder="Johnny"
        />
      </FormControl>
      <FieldDescription>
        This is your public display name.
      </FieldDescription>
      {fieldState.invalid && (
        <FieldError errors={[fieldState.error]} />
      )}
    </Field>
  )}
/>
```

Anatomy of the Controller Pattern

- `<Controller />`: The fundamental bridge between React Hook Form and your UI. It subscribes only to the state of this specific field.
- `render={({ field, fieldState }) => ... }`:
  - `field`: Contains `value`, `onChange`, `onBlur`, and `name`. Essential for **controlled** components.
  - `fieldState`: Provides `invalid`, `isDirty`, and `error` objects for real-time validation feedback.
- `id={field.name}`: Manually ensures that the `Label` correctly targets the `Input` for screen readers.
- `data-invalid`: Allows for easy CSS styling of the entire field wrapper based on the validation state.

Task 3: Upgrade to Shadcn

- **Refactor**: Transform the Task 1 HTML form into a modern, accessible form using the **RHF** `<Controller />` pattern.
- **Components**: Use the custom wrappers provided in `components/ui/field.tsx` (`Field`, `FieldLabel`, `FieldError`).
- **Key Requirements**:
  - **Manual Linking**: Connect `id={field.name}` with the label's `htmlFor` for accessibility.
  - **Type Safety**: Use **Zod Coercion** and `valueAsNumber` for the price field.
  - **Conditional UI**: Show the `discountCode` field only when `hasDiscount` is active using `useWatch`.
- **Goal**: A visually consistent form with explicit validation handling and 100% type safety.

State managment

Why do we need State Management?

- **Server State:** Managing data fetching states: `data`, `loading`, and `error`.
- **UI & Form State:** Handling local interactions, input values, and validation errors.
- **Routing State:** Keeping track of the current route and URL parameters.
- **Data Sharing:** Moving data between components without "prop drilling".
- **Authentication:** Tracking SessionIDs or JWTs to manage user access and identity.

Redux

- **Concept**: A single global "Source of Truth" for the entire application.
- **Principle**: Unidirectional data flow using a store, actions, and reducers.
- **Code Example**:

```
// 1. Action and Reducer
const counterReducer = (state = { count: 0 }, action) => {
  if (action.type === "inc") return { count: state.count + 1 };
  return state;
};
// 2. Dispatching in a component
dispatch({ type: "inc" });
```

- **Downside**: High amount of "boilerplate" code and complex management of asynchronous data.

React Context API

- **Purpose**: Sharing data without manual "Prop Drilling" (passing props through intermediate components).
- **Implementation**:

```
const AuthContext = React.createContext(null);

<AuthContext.Provider value={user}>
  <Navbar /> {/* Accesses 'user' via useContext hook */}
</AuthContext.Provider>;
```

- **Warning**: If the value in the `Provider` changes, every component nested inside it re-renders.

Modern Global State: Zustand

- **Characteristics**: Extremely lightweight, hook-based, and eliminates boilerplate.
- **Implementation**:

```
import { create } from "zustand";

const useCart = create((set) => ({
  items: [],
  add: (item) => set((state) => ({ items: [...state.items, item] })),
}));

// Usage anywhere in the app:
const addItem = useCart((s) => s.add);
```

- **Benefit**: High performance and works outside of the React render cycle.

Atomic State: Jotai / Recoil

- **Concept**: State is split into small, independent pieces called **atoms**.
- **Advantage**: Components subscribe only to specific atoms; changing one atom doesn't affect unrelated components.

```
import { atom, useAtom } from "jotai";

const themeAtom = atom("light");

function ThemeButton() {
  const [theme, setTheme] = useAtom(themeAtom);
  return <button onClick={() => setTheme("dark")}>{theme}</button>;
}
```

Server State: TanStack Query

- **The Problem**: React is not natively designed to manage async data from APIs (loading, errors, caching).
- **The Solution**: Automatic caching and background refetching.

```
const { data, isLoading } = useQuery({
  queryKey: ["todos"],
  queryFn: fetchTodos,
});

if (isLoading) return <p>Loading...</p>;
return (
  <ul>
    {data.map((t) => (
      <li key={t.id}>{t.title}</li>
    ))}
  </ul>
);
```

URL State: TanStack Router

- **Philosophy**: If it is in the URL, it is shareable and survives a page refresh.

```
// URL: /products?sort=name&openId=42
const { sort, openId } = productRoute.useSearch();

return (
  <Link search={{ sort: "price", openId: "10" }}>
    Change filter and open detail
  </Link>
);
```

- **Use Cases**: Filters, sorting, modal states, active tabs, and master/detail views.

Persistence: Browser Storage APIs

- Sometimes, state needs to survive a **page reload** or even a **browser restart**.
- React state is in-memory; once the tab closes, it's gone.
- **Text-only**: They store only text strings. Objects and arrays must be serialized with `JSON.stringify()` and parsed back with `JSON.parse()`.
- **Capacity**: The limit is roughly **5-10 MB** (depends on the browser), which is enough for settings but not for large datasets.
- **Synchronous execution**: Operations run on the **main thread**.
  - Excessive reading/writing of large data volumes can cause UI jank/freezing.

LocalStorage vs. SessionStorage

Although they share the same API (`getItem`, `setItem`), they differ in how long data lives:

Feature	LocalStorage	SessionStorage
Persistence	**Permanent**. Survives closing tab/browser and PC restart.	**Temporary**. Data disappears when the specific tab closes.
Scope	Shared across all windows of the same domain.	Isolated per tab (even within the same domain).
Page refresh	Data remains preserved.	Data remains preserved.
Typical use cases	Dark mode, persistent login, shopping cart.	One-time forms, filters, sensitive session data.

Working with Web Storage in Code

```
// Write an object (requires stringify)
const user = { name: "Adam", role: "admin" };
localStorage.setItem("user_data", JSON.stringify(user));

// Read an object (requires parse)
const storedData = localStorage.getItem("user_data");
if (storedData) {
  const parsedUser = JSON.parse(storedData);
  console.log(parsedUser.name); // "Adam"
}

// Delete
sessionStorage.removeItem("temp_filter");
localStorage.clear(); // Removes absolutely everything in this storage
```

Stop and Think Before You Code

Before picking a tool, ask yourself two critical questions:

1. **Who needs this state and for how long?**
2. **What happens to the performance when this state changes?**

- Does it need to survive a Refresh?
- Does it come from an API?
- Is it just for this component?

Quick Decision Guide

- **Local UI state** (one component): `useState`
- **Complex form state + validation**: React Hook Form (+ Zod)
- **Shared app state**: Context / Zustand / Redux (based on app complexity)
- **Server data**: TanStack Query
- **Shareable filter/sort state**: URL search params / TanStack Router
- **Persistence across refresh**: localStorage or sessionStorage

Summary

- **React Hook Form:** `useForm`, `handleSubmit`, `SubmitHandler<T>`, `register`, `control`, `watch`, `useWatch`, `useFieldArray`
- **Shadcn/UI + RHF:** `<Form />`, `<FormField />`, `<FormItem />`, `<FormControl />`, `<FormMessage />`
- **State Management:** local state, global state, server state, URL state, persistence

That's All for Today!